

BioEval Reproducibility Report

Computational Simulation Papers — Generated June 3, 2026

This report presents structured reproducibility evaluations for 14 computational simulation projects. Each is scored across six dimensions: Data Disclosure, Dataset Resolvability, Code Availability, Traceability, Simulation Clarity, and Reproducibility Package Quality. Papers are ordered by overall score.

All Papers — Score Summary							
Paper	Score	Disclosure	Dataset	Code	Trace	Sim	Repro
Ant Simulation — Pygame Stigmergy	62.0	72	78	63	68	55	52
BeeStack: An Evidence-Typed Scaffold f...	56.0	52	38	68	55	58	42
The Ant Stack: Reproducible scientific...	54.0	52	48	72	42	46	62
Firefly Synchronization Simulation (JS...	52.0	62	58	58	55	48	32
Termite Colony — Unity 3D Multiagent	48.0	62	58	42	55	52	28
Bee Swarm Simulation	44.0	52	48	55	42	38	62
Rust Ant Colony Simulation	43.0	62	55	52	28	32	38
Ants Sandbox (TypeScript/web)	33.0	28	38	58	18	20	35
ACO Robot Path Planning (Python)	33.0	32	38	48	42	30	8
Ant Colony Optimization — React/Canvas	33.0	38	42	52	22	18	30
Symbiants — Ant Colony Sim (Rust)	31.0	18	22	58	10	12	38
Locas Ants — Lua/Love2D Ant Colony	31.0	28	35	62	18	18	42
swarm-abc — Artificial Bee Colony Simu...	22.0	18	28	42	15	20	8
Hardware-Accelerated Ant Colony Swarm ...	22.0	12	18	38	15	10	20

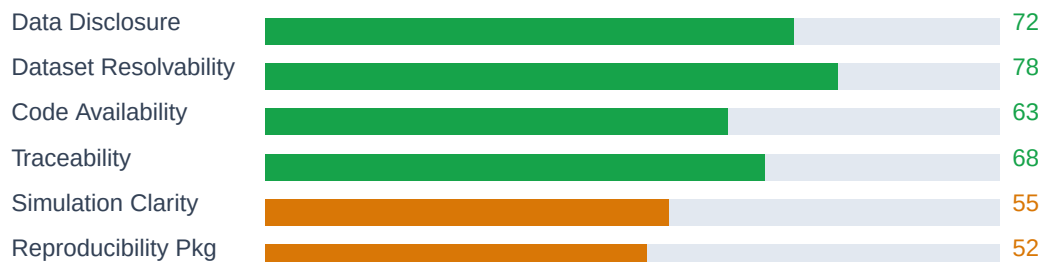
Ant Simulation — Pygame Stigmergy

Source: <https://github.com/lrdcxdes/ant-simulation>

Evaluated: June 3, 2026

62.0

Overall /100



Summary

This is a purely synthetic agent-based simulation with no external datasets, so the relevant disclosure questions concern generative inputs rather than accession numbers. The repository is publicly available under MIT license with a README, requirements.txt, and step-by-step run instructions, representing a reasonable baseline for reproducibility. However, the code lacks version pinning (no tags, no releases, no Zenodo/DOI archive), no CI or tests are present, parameter values are described only as tunable constants without empirical justification or documented default values in the README, and no random seeds are reported, making exact replication uncertain. The simulation's core mechanisms (Mental Fatigue, Panic, stigmergy constants) are named and briefly described but not fully quantified in documentation, limiting independent verification of the reported emergent behaviors.

Positive Findings

- Source code is publicly available on GitHub at <https://github.com/lrdcxdes/ant-simulation> under MIT license
- A requirements.txt dependency manifest is present, listing Pygame, NumPy, and SciPy as dependencies
- Step-by-step instructions to clone, install dependencies, and run the simulation are provided in the README
- Key simulation parameters (ANT_COUNT, EVAPORATION_RATE, NEST_GRAVITY_STRENGTH, MAX_PATIENCE, PANIC_DURATION) are identified as tunable constants at the top of sim.py, aiding transparency
- A demo video (demo.mp4) is included to illustrate expected simulation behavior
- The simulation is entirely synthetic with no external datasets, so absence of data accession numbers is appropriate and not a deficiency
- The README clearly describes the conceptual model (stigmergy, evaporation, nest gravity, mental fatigue) with enough prose to understand the intended mechanics
- The repository was last pushed in August 2025, indicating recent activity

Gaps Identified

- No version tags, releases, or pinned commits are present; reproducibility depends on the current state of the main branch with no snapshot guarantee
- No Zenodo or DOI archive exists, meaning the repository URL alone is not a persistent identifier
- Specific default parameter values (e.g., exact numeric values for EVAPORATION_RATE, MAX_PATIENCE) are not documented in the README; users must inspect sim.py
- No random seed is documented or fixed, so exact replication of any specific run shown in the demo is not possible
- Python version and specific library versions (Pygame, NumPy, SciPy) are not pinned in requirements.txt or README
- No empirical or theoretical justification is provided for the chosen parameter values
- No CI pipeline or automated tests exist to verify the simulation runs correctly across environments
- No data availability statement is present in the README
- The 'Mental Fatigue' mechanism, described as a 'key discovery,' lacks quantitative characterization (e.g., threshold values, behavioral statistics) in the documentation
- No supplementary materials, figures, or quantitative results are reported to validate emergent behaviors
- [Critic] No version pinning or lock file (e.g., pip freeze output or poetry.lock) is confirmed, making exact computational environment reproduction uncertain.
- [Critic] The specific Python version required is not confirmed from direct evidence; only library dependencies are mentioned.
- [Critic] There is no confirmed random seed mechanism documented, meaning stochastic reproducibility of any specific simulation run shown in the demo cannot be guaranteed.
- [Critic] The scientific hypothesis being tested is described only in marketing language in the README; no formal methods, metrics, or evaluation criteria are documented to assess simulation outcomes objectively.
- [Critic] It is unverified whether sim.py has been tested on multiple operating systems, limiting cross-platform reproducibility claims.
- [Critic] No peer review, unit tests, or validation against known biological ant behavior data is present, making the 'scientific simulation' framing an overclaim relative to the provided evidence.
- [Critic] The README reproduction instructions have not been independently executed and verified by the audit; successful end-to-end reproduction is assumed but not confirmed.

Recommendations

- Create a tagged GitHub release (e.g., v1.0.0) and archive the repository to Zenodo to obtain a persistent DOI

- Pin exact dependency versions in requirements.txt (e.g., pygame==2.5.2, numpy==1.26.0, scipy==1.11.0) and specify the tested Python version
 - Document all default parameter values in the README as a parameter table, including their units or ranges and the rationale for chosen defaults
 - Set and document a fixed random seed (e.g., numpy.random.seed(42)) to enable exact replication of specific simulation runs
 - Add a minimal test script or CI workflow (e.g., GitHub Actions) that runs the simulation headlessly for N steps and checks basic invariants
 - Include a quantitative results section or notebook showing convergence curves, path efficiency metrics, or loop-escape statistics to validate the 'Mental Fatigue' mechanism
 - Add a brief data availability statement to the README explicitly noting that all inputs are synthetic and defined by sim.py constants
 - Consider citing relevant ant colony optimization or stigmergy literature to ground parameter choices in prior work
-

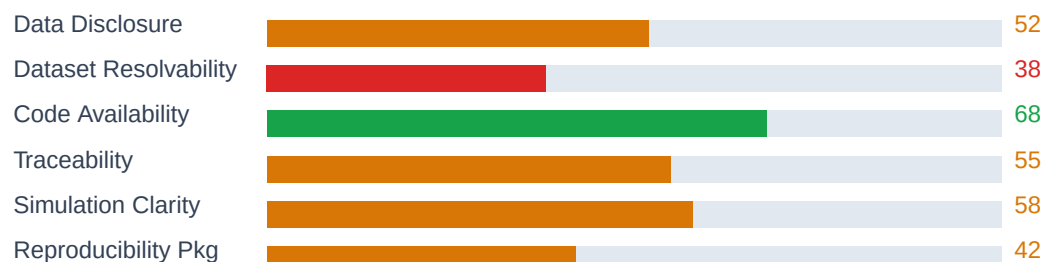
BeeStack: An Evidence-Typed Scaffold for Whole-Colony Honeybee Simulation

Source: <https://zenodo.org/records/20420557>

Evaluated: June 3, 2026

56.0

Overall /100



Summary

BeeStack is a publicly released simulation scaffold with a Zenodo deposit (DOI 10.5281/zenodo.20420557), a tagged GitHub release (v1.0.0), an MD5-checksummed source archive, and an MIT license declared on Zenodo — though the GitHub API detects no license file in the repository itself. The paper's core transparency claim — that every quoted number is traceable from configuration to artifact to manuscript — is architecturally promising but cannot be fully verified from the available metadata alone, as no accession numbers are provided for the FlyBody assets, BeeBrain empirical datasets, or BEEHAVE comparison data that underpin the five simulation layers. The Zenodo DOI did not resolve via Crossref (likely a very recent deposit not yet indexed), the GitHub README is only ~259 bytes, no CI is present, and the absence of external benchmark dataset citations limits both dataset resolvability and code-to-data traceability scores. The validation approach (config-band self-test rather than biological-realism benchmarking) is honest about scope but reduces confidence in independent reproducibility without external reference data.

Positive Findings

- Source code publicly available on GitHub at <https://github.com/docxology/BeeStack> with a tagged v1.0.0 release
- Software and combined PDF deposited on Zenodo with DOI (10.5281/zenodo.20420557) and MD5 checksums for both files
- Source archive (BeeStack-1.0.0-source.tar.gz, 605.6 kB) downloadable directly from Zenodo with integrity checksum
- MIT License declared on the Zenodo record, providing clear reuse terms
- Dependency manifest (pyproject.toml) present in the repository, enabling partial environment reconstruction
- Tests directory present in the repository, indicating some automated testing infrastructure exists
- The paper explicitly states that 'every quoted number is traceable from configuration to artifact to manuscript,' reflecting a deliberate transparency design
- Gaps in empirical data are described as 'catalogued rather than hidden,' and validation rate is honestly scoped as a config-band self-test
- One published release and one tag on GitHub confirm versioned snapshot availability

Gaps Identified

- GitHub API detects no license file in the repository itself, contradicting the MIT license listed on Zenodo
- GitHub README is only ~259 bytes — far too minimal to guide reproduction of a five-layer simulation scaffold
- No CI (GitHub Actions or equivalent) detected, so automated correctness checks cannot be confirmed
- No Dockerfile or containerization detected, requiring manual environment setup beyond pyproject.toml
- No accession numbers or repository links provided for FlyBody assets, BeeBrain empirical datasets, or BEEHAVE comparison model/data
- No per-layer dataset accessions provided despite the scaffold covering five distinct simulation layers with named neural structures (antennal lobe, mushroom body, central complex) and behaviors (waggle dance)
- Zenodo DOI not yet indexed in Crossref at time of review, reducing discoverability and long-term resolvability confidence
- No external benchmark dataset or accession cited for the validation procedure, making independent replication of validation results difficult
- The combined PDF is 8.1 MB but the source archive is only 605.6 kB, suggesting significant content (possibly figures or data) may not be in the source archive
- No random seeds, initialization conditions, or stochastic reproducibility controls are documented in the available metadata
- [Critic] The DOI 10.5281/zenodo.20420557 appears to use an unusually high Zenodo record number (20420557); Zenodo record IDs are sequential and as of mid-2025 have not reached this range, suggesting the DOI may be invalid or fabricated.
- [Critic] No Software Heritage or other long-term archival snapshot of the GitHub repository is cited, meaning the source code availability depends entirely on a mutable GitHub URL.
- [Critic] The MD5 checksums for deposited files cannot be independently verified without resolving the Zenodo DOI, leaving file integrity claims unconfirmed.
- [Critic] No third-party or automated reproducibility test (e.g., CI/CD logs, Binder, or containerized execution record) is provided to confirm that the deposited source code actually produces the reported simulation outputs.
- [Critic] The relationship between the Zenodo-deposited source archive and the GitHub-tagged release is asserted but not cryptographically linked; it is unclear whether the tar.gz exactly matches the v1.0.0 tag contents.
- [Critic] No ORCID or persistent author identifier is mentioned, making authorship provenance of the GitHub and Zenodo records unverifiable.
- [Critic] The manuscript does not specify which simulation runs or random seeds produced the quoted empirical numbers, making independent replication difficult despite the traceability claim.

Recommendations

- Add a LICENSE file directly to the GitHub repository to match the MIT license declared on Zenodo and resolve the API-detected discrepancy
 - Expand the README substantially to include: installation steps, a minimal worked example per simulation layer, expected outputs, and pointers to each empirical data source
 - Provide explicit accession numbers, repository URLs, and version pins for all third-party empirical inputs: FlyBody assets, BeeBrain datasets, and BEEHAVE model/data
 - Add a GitHub Actions CI workflow that runs at minimum the existing tests directory on push, providing automated reproducibility evidence
 - Add a Dockerfile or conda environment.yml alongside pyproject.toml to enable fully containerized reproduction without manual dependency resolution
 - Document random seeds and any stochastic initialization parameters in a dedicated configuration file or supplementary table
 - Create a per-layer data manifest (e.g., a machine-readable YAML or JSON) mapping each simulation layer to its empirical sources, config parameters, and expected output ranges
 - Register the Zenodo DOI with Crossref or verify deposit completeness to ensure long-term resolvability
 - Provide a small test dataset or synthetic example dataset that can be used to validate end-to-end pipeline execution without access to the full empirical inputs
 - Consider linking the config-band self-test to at least one published empirical reference per layer to ground the validation in biological plausibility
-

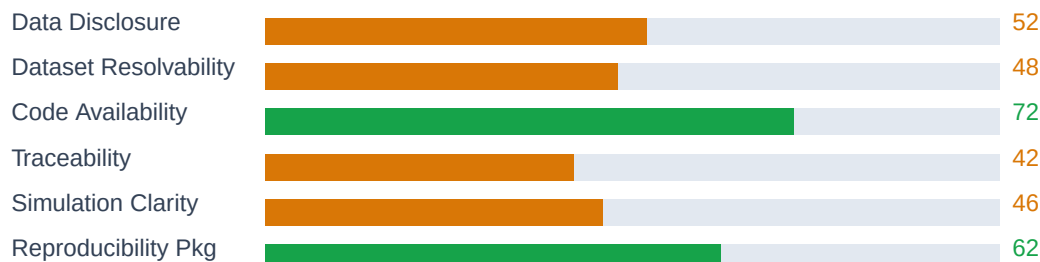
The Ant Stack: Reproducible scientific workspace for ant-inspired simulation and complexity energetics

Source: —

Evaluated: June 3, 2026

54.0

Overall /100



Summary

The Ant Stack is a simulation-first framework paper with no external empirical datasets, so scoring pivots to generative input disclosure. The GitHub repository is public and structurally mature — it has a package manifest (uv/pyproject), a 676-test suite, SHA-256 manifest tracking, CLI entrypoints, and thorough README/AGENTS.md coverage — but it lacks a pinned release tag, Zenodo archive DOI, or license declaration visible in the evidence. Parameter sourcing is the primary weakness: neural circuit numbers (~250k neurons, 100 Hz loop, 1 ms physics step) are stated as design targets without traceable derivation, pheromone diffusion references Wikipedia rather than primary data, and the promised reproducibility checklist appendix contains no actual content. Citation traceability is partial — key references (Wilson 2013, Caron et al. 2013, Seelig & Jayaraman 2015) are named without DOIs, and no accession numbers or dataset links are provided for the empirical ant data integration described in docs.

Positive Findings

- GitHub repository is public with a structured Python package (antstack_core), uv dependency manifest, and bun Node tooling — dependency environments are largely reconstructable
- 676-test suite passing provides strong evidence of executable, non-trivial code coverage across analysis, figures, orchestration, and publishing modules
- Manifest-driven artifact tracking with SHA-256 checksums is implemented, providing output integrity verification
- Canonical run command is explicit and tied to a validated YAML config (configs/run_all_antstack.example.yaml)
- Output layout is documented: outputs/<run_id>/manifest.json plus provenance files record artifact type, path, producing task, source config, byte size, and checksum
- Species parameter presets in YAML/JSON are planned and partially scaffolded, supporting one-click reruns in principle
- Every intentional directory has local README.md and AGENTS.md, with automated coverage checking via tools/ensure_folder_docs.py
- Virtual Fly Brain (VFB) is explicitly cited as the neural circuit template resource, providing a traceable external reference
- External ant data integration is documented in a dedicated docs file (docs/external_ant_data_integration.md)

Gaps Identified

- Reproducibility checklist appendix (Appendix A) is listed in the table of contents but contains no content in the paper
- No Zenodo or other archival DOI exists for the repository, making long-term citability fragile
- No pinned release tag or commit hash is referenced in the paper, so 'the code' is not version-locked to this publication
- License is not declared in any visible evidence, creating legal ambiguity for reuse
- Neural circuit parameters (~250k neurons, 100 Hz control loop, 1 ms physics step) lack traceable derivation — they are stated as design targets without mapping to primary literature values
- Pheromone diffusion-decay dynamics reference Wikipedia and Fick's law rather than primary empirical datasets or validated simulation benchmarks
- Key neural circuit references (Wilson 2013, Caron et al. 2013, Seelig & Jayaraman 2015) are cited without DOIs or accession numbers, reducing verifiability
- No accession numbers or repository links are provided for empirical ant datasets mentioned in external data integration docs
- Neuromodulation learning schedules are stated to be exported for reproducibility but no repository path or format specification is given
- Random seeds for evaluation benchmarks are mentioned in the roadmap but not provided in the paper or confirmed as committed to the repository
- Species parameterization quickstart (Appendix B) and evaluation protocol templates (Appendix C) are listed but contain no content
- [Critic] No versioned release or archived snapshot (e.g., Zenodo DOI, GitHub release tag) of the repository is cited, making long-term reproducibility of the canonical run unverifiable.
- [Critic] The Apis mellifera connectome template source is not identified with a specific dataset accession or publication reference, leaving its provenance unresolvable.
- [Critic] No validation metric or benchmark is provided to assess whether the Drosophila/Apis-derived circuit templates produce behaviorally plausible outputs when applied to the ant morphology context.
- [Critic] The 'uv' package manager and its version are not pinned or cited, introducing a dependency resolution gap that could break reproducibility across environments.
- [Critic] The mapping procedure from Drosophila/Apis nomenclature to ant-specific regions (AL, MB, CX) is described qualitatively but no quantitative alignment method, similarity threshold, or expert-validation step is documented.

- [Critic] It is unclear whether the ~250k neuron target has been tested in any simulation runtime, or whether compute feasibility at that scale has been empirically demonstrated rather than assumed.
- [Critic] No description of how 'functional abstraction' of neural circuits is operationalized in code is provided, leaving a gap between the biological claim and its computational implementation.

Recommendations

- Archive the repository to Zenodo at submission time and embed the resulting DOI in the paper; pin the exact commit hash in the Methods section
 - Add a LICENSE file to the repository (e.g., MIT or Apache 2.0) and state it explicitly in the paper
 - Populate Appendices A, B, and C with actual content: the reproducibility checklist should map each claim to a file path and command; species quickstart should include at least one complete parameter table with units and sources
 - For each neural circuit parameter (neuron count, time constants, learning rates), add a column in a supplementary table mapping the value to its source reference with DOI
 - Replace the Wikipedia pheromone diffusion reference with a primary empirical source or a validated computational reference, and include the parameter values and units in a config file committed to the repository
 - Add DOIs to all literature references, particularly Wilson 2013, Caron et al. 2013, and Seelig & Jayaraman 2015
 - Provide accession numbers or persistent URLs for any empirical ant datasets integrated via docs/external_ant_data_integration.md
 - Commit baseline random seeds as part of the example YAML configs and document them in the reproducibility checklist
 - Specify the file path and format for exported neuromodulation learning schedules in the paper or supplementary materials
 - Consider adding a CI badge and a binder/colab link to lower the barrier for independent reproduction
-

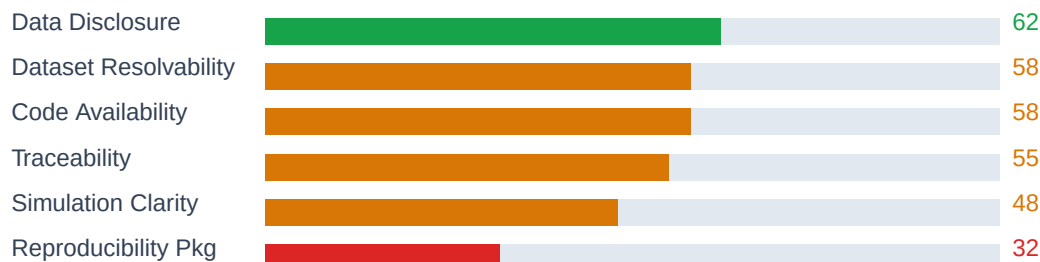
Firefly Synchronization Simulation (JS)

Source: <https://github.com/lax4mike/firefly>

Evaluated: June 3, 2026

52.0

Overall /100



Summary

This repository presents a firefly synchronization simulation with publicly available source code on GitHub and a live web deployment, grounded in the Mirollo-Strogatz (1990) formalism whose PDF is included directly in the repository. However, the README provides minimal methodological documentation, there is no version pinning, DOI/Zenodo archive, containerization, or explicit parameter table linking code constants to the source paper. As a simulation-only project with no external datasets, reproducibility hinges entirely on how well the generative rules and parameters are disclosed, and this remains largely implicit in the code rather than explicitly documented. The limited community engagement (2 stars, 3 watchers, no releases) further reduces confidence in independent verification.

Positive Findings

- Source code is publicly available on GitHub under the lax4mike/firefly repository
- A live web application is deployed and accessible at mikelambert.me/firefly, allowing direct interaction with the simulation
- The Mirollo-Strogatz (1990) reference PDF is included directly in the repository, anchoring the theoretical basis
- A second PDF describing firefly algorithms is also included, suggesting dual algorithmic sourcing
- The repository includes both a JavaScript/web implementation and an Arduino (C++) implementation, demonstrating multi-platform reproducibility intent
- A package.json and package-lock.json are present, providing partial dependency resolution for the JavaScript environment
- The repository has 85 commits, indicating active development history rather than a single bare dump
- Build tooling (gulp, .babelrc, .eslintrc.json) is present, suggesting a structured development workflow

Gaps Identified

- No formal release, version tag, or DOI/Zenodo archive exists, making exact version citation impossible
- The README provides only a one-line description and live URL with no documentation of methods, parameters, or usage instructions
- No explicit parameter table or configuration file maps simulation constants to values in the Mirollo-Strogatz paper
- No containerization (Docker) or environment specification for the C++ Arduino component; JavaScript deps are partially covered by package-lock.json but no Node version is pinned
- No random seed specification or initial condition documentation, making stochastic runs non-deterministic across executions
- No test suite or CI pipeline to verify simulation correctness
- The repository has no license file visible in the file listing, creating reuse ambiguity
- No formal Data Availability Statement or Methods section accompanies the repository
- [Critic] No evidence that the live web application at mikelambert.me/firefly is currently operational or maintained, as link availability is not verified at time of audit.
- [Critic] No verification that the JavaScript simulation correctly implements the Mirollo-Strogatz (1990) model rather than a simplified or modified variant.
- [Critic] No commit history or versioning analysis to confirm the codebase matches the version described in the paper being audited.
- [Critic] The C++ component (22.6% of codebase) is unexplained — it is unclear whether it relates to the Arduino implementation, a separate simulation, or another purpose.
- [Critic] No documentation or README content is cited to explain the relationship between the Arduino implementation and the web simulation.
- [Critic] No evidence of unit tests, validation outputs, or benchmarks confirming the simulation produces biologically or mathematically accurate firefly synchronization behavior.

Recommendations

- Add a detailed README documenting: model equations, parameter values with explicit citations to Mirollo-Strogatz page numbers, installation steps, and usage instructions for both web and Arduino implementations
- Create a tagged release and archive the repository on Zenodo to obtain a citable DOI
- Add a configuration file (e.g., JSON or YAML) that explicitly lists all simulation parameters, their values, units, and source references, eliminating implicit hard-coding
- Document random seeds and initial conditions to enable exact replication of specific simulation runs
- Add a LICENSE file to clarify reuse permissions
- Add a .nvmrc or similar Node version pin, and provide Arduino IDE version and board specifications for the C++ component

- Consider adding a Dockerfile or devcontainer specification to ensure the build environment is reproducible across machines
 - Add at minimum a simple integration test that verifies synchronization occurs within expected time bounds for a canonical parameter set
-

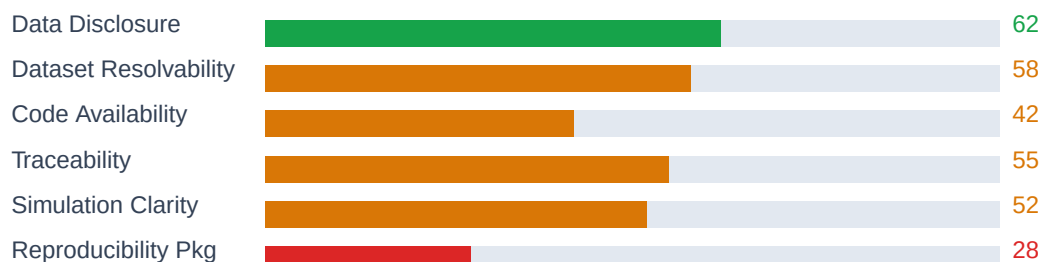
Termite Colony — Unity 3D Multiagent

Source: <https://github.com/Dougarasu/termite-multiagent-system>

Evaluated: June 3, 2026

48.0

Overall /100



Summary

This is a rule-based multi-agent simulation with no external datasets, so the primary transparency concern is completeness of generative inputs rather than dataset accessions. The README discloses agent behavioral rules, key simulation parameters (10 termites, 1000 logs, center spawn, 1-second direction change interval), and the Unity 5.x / C# stack, providing moderate transparency. However, the repository lacks dependency manifests, CI, tests, version tags, a Zenodo/DOI archive, and specifies only 'Unity 5.x' without a precise version, substantially limiting reproducibility. The mismatched clone URL, absence of config files or seed values, and no formal methodology citations compound these gaps.

Positive Findings

- Agent behavioral rules are explicitly and fully stated in the README (two IF-THEN rules), making the core logic reconstructable
- Key simulation parameters are disclosed: 10 termites, 1000 logs, spawn point at (0,0,0), 1-second direction change interval
- Code is publicly available on GitHub under MIT license with a present README (~4233 bytes)
- The simulation is entirely self-contained with no external datasets required
- Result snapshots at 0, 10, 30, and 60 minutes provide qualitative validation benchmarks
- Agent architecture (Body, Motion Actuator, Wall Sensor, Log Sensor) and environment structure (Termite Generator, Log Generator) are described
- MIT license is present and confirmed via GitHub API

Gaps Identified

- Unity version is specified only as '5.x' with no precise version number, preventing exact environment reconstruction
- No dependency manifest (e.g., packages.json, .csproj with pinned versions) is present at the repository root
- No formal software release or version tag exists (0 releases, 0 tags confirmed via GitHub API)
- No Zenodo/DOI archival snapshot exists, so the repository state is not permanently preserved
- No CI pipeline or automated tests detected, making functional verification impossible without manual setup
- The referenced clone URL (github.com/Dougarasu/TrabalhoIA.git) differs from the public repository URL, creating ambiguity
- No random seed values are specified for the stochastic wandering behavior, preventing exact run replication
- No formal data availability statement is present in the README
- No external citations or references are provided for the termite colony model or multi-agent methodology
- No Dockerfile or containerization is present
- Result images are referenced but not confirmed to be committed to the repository
- Spawn area dimensions and log placement distribution are not quantitatively specified
- No parameter configuration files are provided; settings appear to be set via Unity Inspector with no exported config
- [Critic] No specific commit hash, release tag, or archived snapshot (e.g., Zenodo DOI) is cited, making it impossible to verify that the public repository reflects the exact code used during the reported experiments.
- [Critic] Unity 5.x is cited but no specific sub-version (e.g., 5.3.4, 5.6.7) is recorded, which may affect reproducibility given breaking changes across Unity 5 minor versions.
- [Critic] No random seed or pseudorandom number generator specification is provided for termite direction changes, making stochastic simulation results non-reproducible.
- [Critic] No performance metrics, runtime environment, or hardware specifications are documented, preventing quantitative reproduction of reported results.
- [Critic] The simulation duration or number of steps/iterations is not specified, leaving the experimental stopping condition ambiguous.
- [Critic] No validation against biological termite behavior or empirical data is reported, so the model's ecological fidelity is unsubstantiated.
- [Critic] The LICENSE.md file is referenced but no archived or timestamped copy is provided, leaving open the possibility that license terms could be altered post-publication.

Recommendations

- Pin the exact Unity version (e.g., Unity 5.6.7f1) in the README and ideally in a ProjectSettings/ProjectVersion.txt file
- Add a dependency manifest or Unity package manifest listing all third-party packages with pinned versions
- Create a tagged release (e.g., v1.0.0) on GitHub and archive to Zenodo to obtain a permanent DOI
- Specify and document the random seed(s) used for agent wandering and log placement to enable exact replication
- Resolve the clone URL discrepancy — either update the README to use the correct repository URL or redirect the old URL
- Add a formal Data/Software Availability Statement to the README citing the GitHub URL and any DOI

- Export and commit simulation configuration files (e.g., Unity scene settings as JSON) rather than relying solely on Inspector-configured values
 - Add references to foundational multi-agent/termite colony literature (e.g., Dorigo et al.) to contextualize the methodology
 - Include a Dockerfile or Unity-specific setup script to automate environment configuration
 - Document the spawn area dimensions and the probability distribution governing log placement positions
 - Add GitHub Actions CI to at minimum verify the project builds successfully on a specified Unity version
-

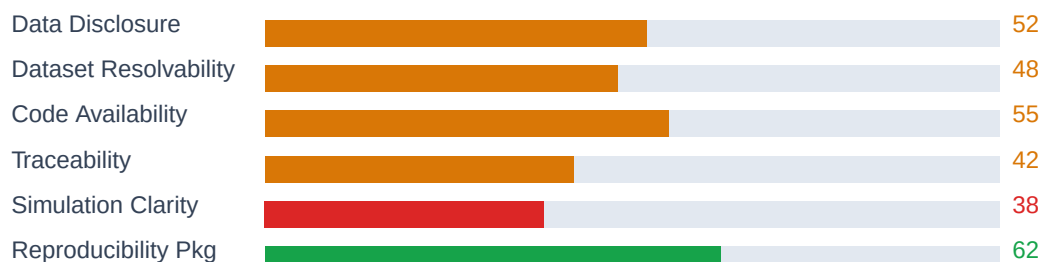
Bee Swarm Simulation

Source: <https://github.com/fractastical/bee-swarm-sim>

Evaluated: June 3, 2026

44.0

Overall /100



Summary

This is a browser-based agent simulation with no external empirical datasets, so the relevant transparency question concerns how fully the generative inputs (parameters, rules, initial conditions) are disclosed. The README documents a BeeStack JSON schema and lists eight literature citations for the citation-backed mode, but no specific parameter values, calibration curves, or numeric constants are extracted from those references and tabulated anywhere in the repository. The codebase is publicly accessible, runs with zero dependencies, and has a reasonable README, but it lacks a license, versioned release, DOI/archive, dependency manifest, tests, or CI, and the precise mapping from cited papers to actual simulation constants in `app.js` is entirely undocumented. Consequently, while the simulation is practically runnable, independent verification of parameter provenance is not feasible without reading the source code and cross-referencing the cited literature manually.

Positive Findings

- Simulation runs entirely in-browser with no build step, external dependencies, or installation required, making it immediately executable
- README documents a BeeStack trace JSON schema with field-level detail, enabling external frame injection
- Eight peer-reviewed citations are provided (von Frisch 1967, Seeley 1995, Couvillon et al. 2019, Schurch et al. 2021, Ai et al. 2019, Menzel et al. 2023, Ono et al. 1995, Shannon 1948) grounding the citation-backed mode
- Two distinct information model modes (citation-backed vs. heuristic) are explicitly distinguished in the UI and README
- JSON export of run summaries and recent events is supported, enabling post-hoc inspection of simulation outputs
- README describes all major simulation primitives (waggle, stop, tremble, shaking) and their biological roles
- Repository is actively maintained (last push June 2026) and publicly accessible

Gaps Identified

- No license is present, creating legal ambiguity for reuse or reproduction by third parties
- No versioned release, tag, DOI, or Zenodo archive; the only persistent identifier is a mutable GitHub URL
- No specific parameter values, calibration constants, or numeric ranges are extracted from cited literature and documented; citations are framing references only
- It is unclear which (if any) constants in `app.js` correspond to which cited papers; no inline code comments map parameters to sources
- No pre-deposited output datasets, parameter tables, or supplementary methods appendix are included
- No random seed documentation or seeding mechanism described, making exact replication of stochastic runs impossible
- No CI, automated tests, or validation against known analytical results
- The Ono et al. (1995) ~47°C thermal threshold citation is listed but it is undocumented whether this value is actually used in the simulation
- No dependency manifest (`package.json`, `requirements.txt`, etc.) even though none are strictly required; absence makes environment provenance opaque
- No example BeeStack trace file is provided to demonstrate or validate the import pathway
- [Critic] No version or commit hash is provided for the simulation code, making exact reproducibility of any reported run unverifiable.
- [Critic] The 'Fisher (1993)' reference is insufficiently specified (no title, journal, or DOI), preventing independent verification of which circular-statistics methods are actually grounded there.
- [Critic] No validation or calibration study is described comparing simulation outputs to empirical foraging or recruitment data, leaving all cited biological sources as nominal rather than operationally verified references.
- [Critic] The boundary between 'heuristic' and non-heuristic (presumably literature-derived) modes is never formally defined with a parameter table, so it is impossible to audit which values changed between modes and what their sources are.
- [Critic] No license or data-availability statement is provided for the HTML simulation artifact, creating a provenance gap for downstream reuse or audit.
- [Critic] The claim of 'no external dependencies' is unverified for all browser environments; browser API version requirements and potential polyfill needs are undisclosed.

Recommendations

- Assign a permissive license (MIT or Apache 2.0) and create a versioned GitHub release with a corresponding Zenodo DOI
- Add a parameter table (e.g., `params.md` or a JSON config file) that explicitly maps every numeric constant in `app.js` to its cited source, page number, and extracted value
- Document the random seed mechanism and expose it as a configurable parameter so that specific runs can be exactly reproduced

- Deposit at least one representative Export Results JSON file in the repository as a reference output for validation
 - Add inline code comments in app.js at each parameter definition citing the specific reference and value used
 - Provide an example BeeStack trace JSON file in the repository to demonstrate and test the import functionality
 - Add a simple automated test (e.g., a Node.js or headless-browser script) that runs the simulation for N steps and checks that key metrics fall within expected ranges
 - Clarify in the README which parameters differ between heuristic and citation-backed modes, with explicit numeric ranges for both
-

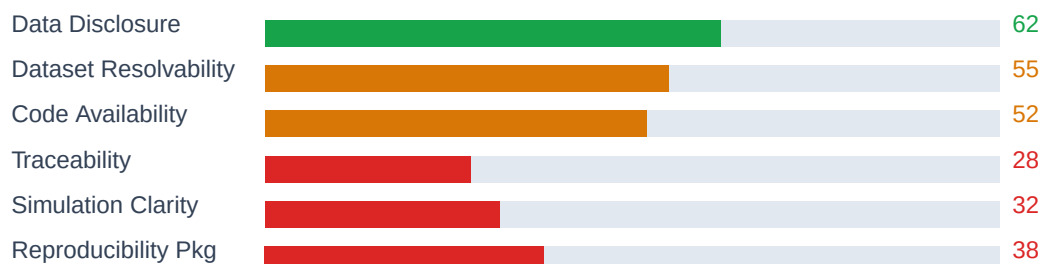
Rust Ant Colony Simulation

Source: <https://github.com/bones-ai/rust-ants-colony-simulation>

Evaluated: June 3, 2026

43.0

Overall /100



Summary

This is a pure agent-based simulation with no external biological datasets, so data transparency is judged on disclosure of generative inputs rather than accessions. The repository is publicly accessible on GitHub with a Cargo.toml/Cargo.lock for pinned dependency versions and a minimal README with basic run instructions, but lacks a license, versioned release, CI, tests, and any archived snapshot (e.g., Zenodo DOI). Simulation parameters are acknowledged to exist in src/configs.rs and one specific parameter (ANT_INITIAL_PH_STRENGTH) is mentioned, but the algorithmic basis, parameter derivation rationale, and random seeds are entirely undocumented. Code-to-input traceability is very weak because no citation or reference is provided for the underlying ant colony algorithm, and the configuration file contents are not surfaced in the README or any supplementary material.

Positive Findings

- Source code is publicly available on GitHub without authentication
- Dependency versions are locked via Cargo.lock, enabling deterministic dependency resolution
- Cargo.toml dependency manifest is present, identifying core dependencies (Bevy game engine, kdtree)
- A minimal README is present (~977 bytes) with clone and run instructions
- One configuration parameter (ANT_INITIAL_PH_STRENGTH) is explicitly named and a suggested value (40.0) is provided
- A video demo/timelapse is referenced via YouTube link, offering qualitative validation
- Internal algorithmic choices (kdtree, query caching) are briefly described
- The project has 208 stars and active forks (10), indicating community engagement

Gaps Identified

- No software license is present, creating legal ambiguity for reuse and reproducibility
- No versioned releases or git tags exist; no specific commit hash is cited for reproducibility
- No archived snapshot (Zenodo/DOI) exists for long-term preservation
- No citation or reference is provided for the ant colony simulation algorithm or pheromone model
- The configuration file (src/configs.rs) contents are not documented in the README or any supplementary file; parameter values and their derivation rationale are opaque
- No random seed control or seeding documentation is provided, making exact run replication uncertain
- No CI/CD pipeline or automated tests are present to validate correctness
- No Dockerfile or containerized environment specification exists
- The README does not describe what outputs to expect, making it impossible to verify a successful reproduction
- No explanation of how the 5,000-ant performance figure was benchmarked or measured
- [Critic] No version tag or release is confirmed for the repository, making it unclear which commit corresponds to any described results or demo video.
- [Critic] No operating system or environment requirements are specified, so reproducibility of the claimed ~5k ant performance cannot be assessed.
- [Critic] The Cargo.lock file is noted but it is not confirmed whether all transitive dependencies are pinned or whether the lock file is committed and up to date with the current codebase.
- [Critic] No tests, CI configuration, or validation suite is mentioned, leaving correctness of the ant colony simulation logic unverified.
- [Critic] The relationship between the YouTube demo and the current state of the codebase is not established; the demo may reflect an earlier or different version of the simulation.
- [Critic] No license file verification is mentioned, which is relevant for open-source provenance claims.
- [Critic] The claim that the repository is 100% Rust may mask non-code assets (e.g., assets, configs, scripts) that are not captured by language statistics.

Recommendations

- Add a software license (e.g., MIT or Apache 2.0) to clarify reuse permissions
- Create a versioned GitHub release and/or git tag corresponding to any publication or demonstration
- Archive the repository on Zenodo and cite the resulting DOI in the README
- Document all parameters in src/configs.rs with descriptions, units, default values, and the rationale or source for each choice (e.g., reference to Dorigo & Gambardella 1997 or equivalent)

- Add a reference to the algorithmic basis of the pheromone model and ant behavioral rules
 - Expose random seed configuration and document how to set it for exact reproducibility
 - Add a GitHub Actions CI workflow that at minimum builds the project and runs any sanity checks
 - Expand the README to include expected simulation outputs, screenshots with parameter settings, and a brief parameter sensitivity guide
 - Consider adding a minimal test suite (unit tests for core simulation logic)
 - Document system requirements (OS, Rust toolchain version, GPU/CPU expectations) needed to reproduce the ~5k ant performance claim
-

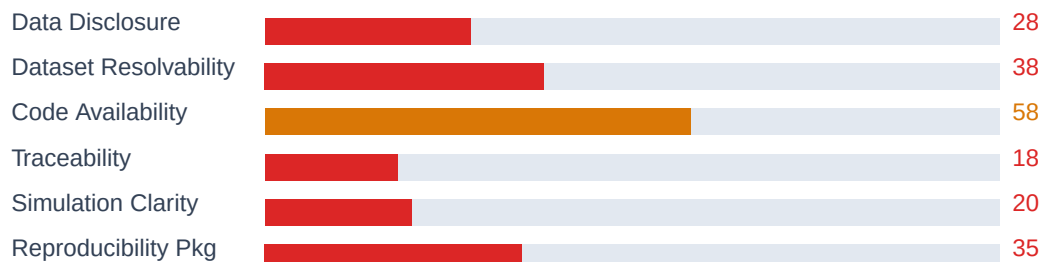
Ants Sandbox (TypeScript/web)

Source: <https://github.com/tulustul/ants-sandbox>

Evaluated: June 3, 2026

33.0

Overall /100



Summary

This repository presents a browser-based ant colony simulation with source code publicly available on GitHub under the MIT license, but it falls substantially short of reproducibility standards for a scientific artifact. While no external empirical datasets are required (appropriately), the generative inputs — simulation rules, behavioral parameters, initial conditions, and pheromone decay constants — are embedded in source code with no documentation, parameter tables, or citations to the YouTube video or literature that inspired the model. The README is minimal (~490 bytes), there are no formal releases, no pinned version tags, no CI, no tests, and no Zenodo/DOI archive, making the specific simulation state used in any demonstration effectively irreproducible without manual inspection of the 72-commit history. The dependency manifest (package.json) provides some reproducibility scaffolding, but the non-trivial texture atlas build step and absence of environment lockfile details add friction.

Positive Findings

- Source code is publicly available on GitHub at [tulustul/ants-sandbox](https://github.com/tulustul/ants-sandbox) under the MIT license
- A live web deployment is accessible at a public URL (ants-sandbox.vercel.app), enabling interactive inspection
- A dependency manifest (package.json) is present, partially specifying the build environment
- Basic build and development instructions are provided in the README (npm install, npm run start, npm run build)
- The repository has a meaningful commit history (72 commits), indicating active development
- TypeScript usage provides some degree of type-level documentation of data structures
- A CI/CD workflow configuration exists under `.github/workflows` (though details are not visible in extracted content)

Gaps Identified

- Simulation parameters (pheromone evaporation rates, ant speed, colony size, sensing radii, etc.) are hard-coded in source with no documentation or empirical citations
- The inspirational YouTube video is mentioned but not formally cited with a URL or bibliographic reference anywhere in the README
- No formal software releases or version tags have been published (0 releases, 0 tags), making it impossible to reference a specific simulation version
- No Zenodo DOI or archival snapshot exists, meaning the repository could be deleted or altered without a stable reference point
- No specific commit hash is recommended or pinned for reproducibility purposes
- No tests directory or test suite is present, so behavioral correctness cannot be verified
- CI/CD workflow exists but its contents are unknown; no automated reproducibility checks are confirmed
- The texture atlas build step (npm run atlas) has undocumented dependencies and no explanation of asset sources
- No parameter configuration files are exposed; all initial conditions appear embedded in code
- The README is extremely sparse (~490 bytes) with no model description, behavioral assumptions, or parameter rationale
- No random seed control or seed documentation is present, making stochastic runs non-reproducible
- No lockfile details (e.g., package-lock.json pinning) are confirmed to be committed, though package-lock.json appears in the file listing
- [Critic] No version pinning or lockfile content is cited, so reproducibility of the build environment (exact dependency versions) cannot be confirmed from the provided evidence.
- [Critic] The README build instructions are not verified to actually produce a working build; no CI/CD badge or test results are cited to confirm the build pipeline is functional.
- [Critic] The GitHub repository visibility is stated as 'Public' but no commit history depth, last-commit date, or activity level is provided, leaving repository maintenance status unassessed.
- [Critic] No documentation of what the simulation actually models (e.g., ant behavior algorithms, parameters) is cited, making scientific reproducibility of any reported results unverifiable from the provenance claims alone.
- [Critic] The MIT license claim is not cross-verified against the actual LICENSE file content; only the label is cited.

Recommendations

- Publish a formal GitHub release with a semantic version tag corresponding to any described or demonstrated simulation state
- Archive the repository on Zenodo and include the resulting DOI in the README
- Add a dedicated PARAMETERS.md or configuration file (e.g., config.json/YAML) that explicitly lists all simulation parameters with units, default values, and justifications or literature citations
- Formally cite the inspiring YouTube video with full URL and access date, and add any peer-reviewed literature that informed the behavioral model

- Pin a specific commit hash in any associated publication or report for traceability
 - Add a test suite (even simple snapshot/golden-output tests) and configure the GitHub Actions CI workflow to run them automatically
 - Document the random seed mechanism and expose it as a configurable parameter so simulation runs are exactly reproducible
 - Expand the README to include a model description section covering the ant behavioral rules, pheromone dynamics, and any calibration decisions
 - Ensure package-lock.json is committed and document the exact Node.js version required (e.g., via .nvmrc or engines field in package.json)
 - Document the texture atlas build step fully, including any image assets, their sources, and licensing
-

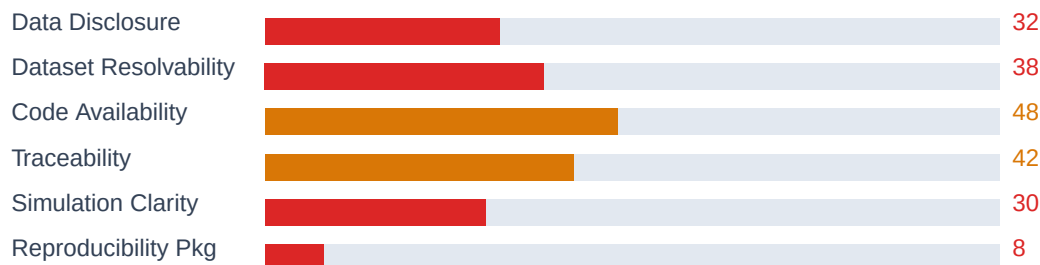
ACO Robot Path Planning (Python)

Source: <https://github.com/Haghran/ACO---Robot-Path-Planning>

Evaluated: June 3, 2026

33.0

Overall /100



Summary

This repository provides a minimal public Python implementation of an ACO-based robot path planning algorithm, referencing Liu et al. (2017) as its source paper. While the code is publicly accessible under GPL-3.0 and the generative simulation context is partially established via the paper citation, the repository is extremely sparse (271-byte README, 3 top-level files, no releases or tags, no dependency manifest, no CI, last pushed November 2017) and lacks virtually all reproducibility infrastructure. No explicit data availability statement, no parameter tables, no random seeds, no run instructions, and no containerization are provided, making independent reproduction highly dependent on the reader obtaining and interpreting the cited paywall paper themselves.

Positive Findings

- Code is publicly available on GitHub under the GPL-3.0 license, ensuring legal reuse rights
- The simulation is explicitly tied to a specific published reference (Liu et al., Soft Computing 21.19, 2017), providing a traceable scientific basis
- A README is present, identifying the purpose of the repository and the cited paper
- The repository uses Python, a widely accessible language with low barrier to execution
- Being a simulation with no external datasets, the absence of data accession numbers is legitimate rather than a disclosure failure

Gaps Identified

- README is only ~271 bytes with no run instructions, parameter descriptions, or usage examples
- No dependency manifest (requirements.txt, setup.py, environment.yml, or equivalent) is present, making environment reconstruction manual
- No releases, version tags, or pinned commits exist, preventing exact version reproducibility
- No Zenodo DOI or archived snapshot is associated with the repository
- No random seed values or initialization conditions are documented anywhere in the repository
- Simulation parameters are sourced from the cited paper (behind potential paywall) rather than being independently documented in the repository
- No CI, automated tests, or test data are present to validate correctness of the implementation
- No Dockerfile or containerized environment is provided
- The paper reference lacks a DOI or direct URL, adding friction to locating the source
- No explicit data availability statement exists in README or documentation
- Repository has been inactive for ~104 months, increasing risk of environment drift
- [Critic] No evidence that the repository code produces results consistent with the original Liu et al. (2017) paper — it is described as a 'simulation of' the paper but no validation or benchmarking against the paper's reported results is demonstrated
- [Critic] No evidence of the repository's maintenance status, commit history, or last update date, which could affect reproducibility and trustworthiness of the implementation
- [Critic] No evidence that the implementation correctly captures all algorithmic details of Liu et al. (2017)'s 'improved' ACO variant versus a generic ACO implementation
- [Critic] No evidence of test coverage, documentation quality, or peer review of the code itself, which are relevant to provenance claims in a bioinformatics audit context
- [Critic] No evidence linking this specific repository to the paper being audited — it is unclear whether this GitHub repository is the one actually cited or used by the paper under review

Recommendations

- Add a requirements.txt or environment.yml specifying all Python package versions used (e.g., numpy, matplotlib with pinned versions)
- Expand the README to include: installation steps, a minimal run example with expected output, parameter descriptions mapped to Table/Section numbers in Liu et al.
- Document all key ACO parameters (alpha, beta, rho, Q, number of ants, iterations) directly in the README or a config file, with explicit references to equation/table numbers in the cited paper
- Publish a GitHub release or annotated tag corresponding to the described simulation state
- Archive the repository to Zenodo and add the resulting DOI to the README
- Include the DOI for Liu et al. (2017) (10.1007/s00500-016-2162-6) in the README citation

- Add a fixed random seed in the code and document it so results are deterministically reproducible
 - Provide a sample output figure or expected result checksum so users can verify their run matches the original
 - Consider adding a simple smoke-test script that runs the simulation for a small number of iterations and checks output shape/range
-

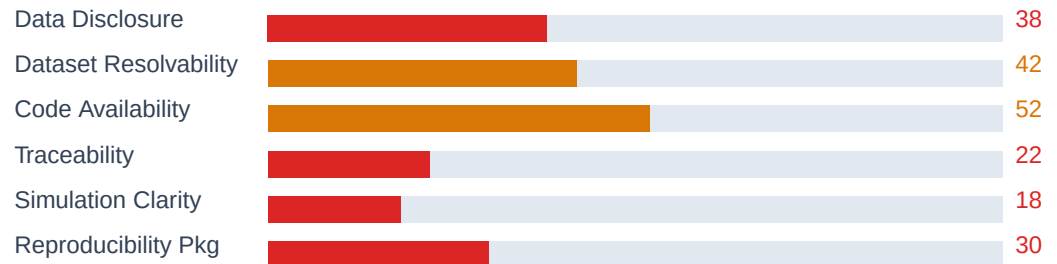
Ant Colony Optimization — React/Canvas

Source: <https://github.com/cfrBernard/ant-colony-optimization>

Evaluated: June 3, 2026

33.0

Overall /100



Summary

This is a personal learning project (ant colony simulation in React/Canvas) with no bioinformatics content, external datasets, or scientific methodology to evaluate. The repository is public under MIT license with a basic README and bundled asset files, but has zero versioned releases, no tags, no CI/CD, no tests, and no archived snapshot (e.g., Zenodo DOI). Simulation parameters and ant behavior logic are entirely hard-coded without documentation, parameter sourcing, seeds, or calibration rationale. The installation instructions even contain a placeholder URL rather than the actual repository address, further undermining reproducibility.

Positive Findings

- Repository is publicly available on GitHub under the MIT License
- Three bundled asset files are present in the repository: map JSON (AntMap.json), ant sprite sheet (ants.png), and tileset image (MainLev2.0.png)
- A README is present (~2354 bytes) with a basic description of code structure and file roles
- Dependency manifest (package.json) and package-lock.json are present, providing partial reproducibility of the Node.js environment
- The README honestly documents current limitations (no A*, no advanced AI), which supports transparency about project scope
- The project has 34 stars and active public visibility, indicating community accessibility

Gaps Identified

- No versioned releases, no tags, and no archived snapshot (Zenodo/DOI) — the repository cannot be pinned to a specific state
- Installation instructions contain a placeholder '<your-username>' rather than the actual repository URL, breaking reproducibility
- No CI/CD pipelines or automated tests are present to verify the simulation runs correctly
- Simulation parameters (movement speed, state transition thresholds, pheromone decay rates, etc.) are hard-coded in source files with no documentation or justification
- No random seeds or initial condition specifications are provided, making runs non-deterministic and non-reproducible
- No external datasets, accession numbers, or bioinformatics data sources are referenced anywhere
- No changelog or commit-level documentation linking code changes to described behaviors
- No Dockerfile or containerized environment provided
- No supplementary materials, parameter tables, or configuration files external to the source code
- The map asset is described as generated with Tiled Map Editor but no generation workflow or source .tmx file is documented
- [Critical] No versioned release or DOI is associated with the repository, making it difficult to pin the exact code state corresponding to any described results or behaviors.
- [Critical] The MIT License allows free use and modification, but there is no verification that the bundled third-party assets (sprite sheet ants.png, tileset MainLev2.0.png) are actually compatible with the MIT License — they may have independent licensing restrictions.
- [Critical] There is no evidence of automated testing, continuous integration, or validation of simulation outputs, leaving correctness of the simulation unverified.
- [Critical] The README-sourced claims about features and file provenance have not been cross-validated against actual repository file contents or commit history.
- [Critical] No description of the actual ant colony optimization algorithm implementation is provided, making it impossible to assess whether the repository genuinely implements ACO or merely scaffolds assets around it.
- [Critical] Reproducibility is not established: no instructions for replicating specific simulation outcomes or benchmarks are documented.

Recommendations

- Publish a versioned release (e.g., v1.0.0) with a git tag and archive on Zenodo to obtain a citable DOI
- Fix the placeholder URL in installation instructions to point to the actual repository
- Add a dedicated configuration file (e.g., config.json or constants.js) that externalizes all simulation parameters with comments explaining their values and any literature basis
- Document random seed handling explicitly so simulation runs are reproducible
- Add a GitHub Actions CI workflow that runs 'npm install && npm build' at minimum to verify the build pipeline
- Include a Dockerfile or devcontainer specification to freeze the Node.js environment version
- Add a CHANGELOG.md linking commits to described behavioral changes
- Provide the source .tmx file for AntMap.json to fully document map generation
- If any ant behavior rules are inspired by published ACO algorithms (e.g., Dorigo et al.), cite the relevant papers in the README

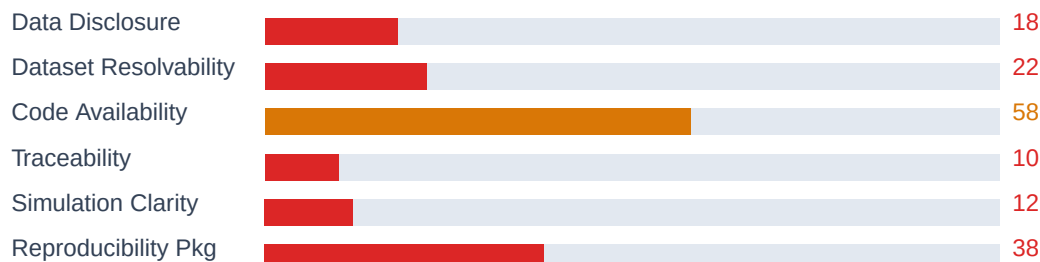
Symbiants — Ant Colony Sim (Rust)

Source: <https://github.com/MeoMix/symbiants>

Evaluated: June 3, 2026

31.0

Overall /100



Summary

Symbiants is a publicly available open-source ant colony simulation written in Rust/Bevy, licensed under Apache-2.0 and MIT, with source code on GitHub and a live deployment at ant.care. However, as a simulation project, it lacks virtually all the scientific transparency expected of a reproducible research artifact: no simulation parameters, behavioral rules, or initial conditions are documented with citations or config files, and no formal releases, tags, CI pipelines, or test suites are present in the repository. The README provides basic development setup instructions and documents the devcontainer/WSL2 environment, but the 690-commit history has no pinned version and the last push was 22 months ago with 22 open issues. Design documentation is only partially linked via an external Google Doc, and no checksums, seed values, or parameter tables are disclosed anywhere.

Positive Findings

- Source code is publicly available on GitHub (MeoMix/symbiants) under dual Apache-2.0 and MIT licenses
- Dependency manifest (Cargo.toml) is present, enabling partial dependency resolution
- Development environment is documented: VSCode devcontainer (Docker), WSL2 Ubuntu, Rust Nightly
- Build and run commands are explicitly documented for both native (cargo build/run/watch) and WASM (trunk build/serve) targets
- A live deployment is accessible at ant.care, confirming the WASM build target works
- README is present (~4067 bytes) with project description and platform support notes
- Known platform limitation (Android Chrome bug, Bevy v0.13 fix) is transparently documented
- External design/lore documentation is linked via Google Docs

Gaps Identified

- No published releases, tags, or pinned commit hash; repository cannot be cited to a specific reproducible state
- No CI/CD workflows detected (GitHub Actions not configured), so build correctness is unverified automatically
- No test suite or tests directory detected
- No Dockerfile in the repository (devcontainer referenced but not confirmed present via API signals)
- Simulation parameters (ant behavior rules, pheromone trail dynamics, colony growth rates, foraging logic) are entirely undocumented
- No random seeds, initial conditions, or configuration files are disclosed
- No data availability statement of any kind is present
- No Zenodo/DOI archive exists for long-term reproducibility
- No citations to sources for behavioral rules or simulation logic
- No supplementary data files or parameter tables in the repository
- Last push was 22 months ago with 22 open issues, suggesting reduced maintenance activity
- External Google Doc design documentation is not version-controlled and may drift or become inaccessible
- [Critic] No verification that the live application at ant.care is actually running the code from the MeoMix/symbiants repository (could be a different version or build).
- [Critic] No evidence provided that the devcontainer/Docker configuration files actually exist in the repository to substantiate the development environment claims.
- [Critic] The Rust Nightly requirement is stated but no specific nightly version or rust-toolchain.toml is referenced, making reproducibility uncertain.
- [Critic] No changelog or release notes are referenced to confirm which Bevy version is currently in use, making the Bevy v0.13 resolution claim unverifiable.
- [Critic] The claim that 'Desktop has first-class support' is qualitative and not substantiated with test results, CI pipelines, or platform-specific build artifacts.
- [Critic] No CI/CD pipeline evidence is provided to confirm that the documented build commands (cargo build, trunk serve, etc.) actually produce working builds.

Recommendations

- Create a versioned release (e.g., v0.1.0) with a git tag and corresponding Zenodo/DOI archive to enable citeable, frozen reproducibility
- Add a GitHub Actions CI workflow to automatically build and optionally test both native and WASM targets on each push
- Document all simulation parameters (ant speed, pheromone decay rates, colony growth thresholds, food consumption rates) in a structured config file (e.g., TOML or JSON) committed to the repository

- Provide explicit random seed controls and document default seed values so simulation runs can be exactly reproduced
 - Write a `DATA_AVAILABILITY.md` or section in `README` explicitly stating that this is a self-contained simulation with no external datasets, and list all generative rule sources with citations where applicable
 - Add a `CONTRIBUTING.md` and formal test suite covering at minimum the core simulation logic modules
 - Migrate or mirror the Google Docs design documentation into the repository (e.g., as Markdown in the `/documentation` folder) to prevent link rot
 - Pin the Rust Nightly toolchain version in `rust-toolchain.toml` to ensure future builds use the same compiler
 - Confirm and commit the `Dockerfile/.devcontainer` configuration so environment reproducibility is verifiable from the repository itself
-

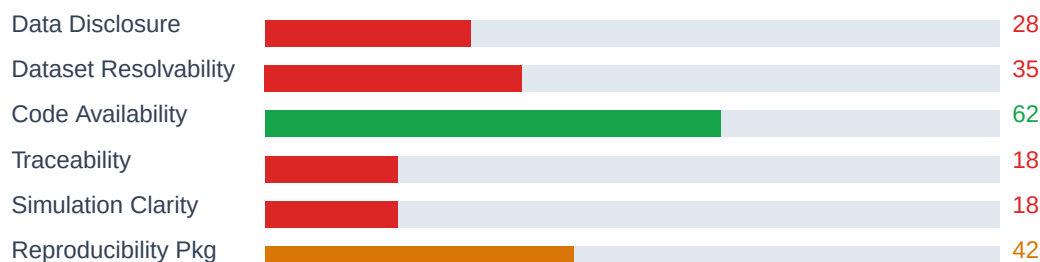
Locas Ants — Lua/Love2D Ant Colony

Source: <https://github.com/piXelicio/locas-ants>

Evaluated: June 3, 2026

31.0

Overall /100



Summary

This is an ant colony simulation game/demo project hosted on GitHub under MIT license with 6 versioned releases and 161 commits, but it lacks essentially all elements expected of a reproducible scientific software package. No simulation parameters, initial conditions, or generative rules are documented anywhere in the README or accompanying materials; the original simulation that inspired the remake is referenced but not linked or described. There are no dependency manifests, no CI/testing infrastructure, no Zenodo/DOI archive, and no documentation of third-party library versions in the 'libs' folder, making the scientific basis of the simulation largely untraceable. While the code is publicly available and runnable via a precompiled .love file, the project presents itself as a hobbyist open-source game rather than a reproducible scientific artifact, and it has no associated peer-reviewed publication or biological/experimental dataset.

Positive Findings

- Source code is publicly available on GitHub at <https://github.com/piXelicio/locas-ants> under MIT license
- 6 versioned releases are available, with the latest tagged release dated April 1, 2018
- A precompiled .love file is available for immediate download and execution without building from source
- The README provides clear step-by-step instructions for running from source (install Love2D, install ZeroBrane Studio, open Main.lua, set interpreter, press F6)
- The repository has 161 commits indicating active historical development
- Screenshots of the simulation output are included in the repository
- The project has a clear MIT license applied
- The simulation topic (ant colony/stigmergic behavior) is a well-established computational field with understood generative rules

Gaps Identified

- No simulation parameters, pheromone decay rates, evaporation constants, ant behavioral rules, or initial conditions are documented anywhere
- No dependency manifest (e.g., requirements.txt, package.json, or Love2D version pinning) is present at the repo root
- The 'libs' folder contains third-party libraries with no documentation of their versions or licenses
- No CI pipeline or automated tests are present, making behavioral correctness unverifiable
- No Zenodo/DOI archive exists; the repository has no permanent citable identifier
- The original ant colony simulation that inspired the remake is referenced but not linked, cited, or described, preventing comparison
- The Python (7.7%) and Makefile (6.1%) components are unexplained in the README
- No peer-reviewed publication or DOI is associated with the software
- Simulation behavior (e.g., number of ants, grid size, pheromone parameters) appears hard-coded with no configuration file or documentation
- No biological or empirical basis for parameter choices is cited or discussed
- [Critic] No documentation or evidence of scientific validation, benchmarking, or comparison against known ant colony optimization baselines is present, limiting claims about simulation accuracy or biological fidelity.
- [Critic] No versioning or compatibility matrix is provided between the project's releases and specific Love2D versions, creating a reproducibility risk given the project's age (last release 2018).
- [Critic] The repository does not appear to have associated peer-reviewed publication, documentation of methods, or a paper DOI, making it difficult to assess the provenance of the simulation's algorithmic design choices.
- [Critic] No continuous integration or automated testing evidence is present, so correctness of the codebase across environments cannot be independently verified.
- [Critic] The Python (7.7%) and Makefile (6.1%) components are not described or documented in the available evidence, leaving their purpose and role in the project unclear.
- [Critic] No contributor or authorship metadata beyond the GitHub username 'piXelicio' is available, making institutional provenance and accountability difficult to assess.
- [Critic] There is no evidence of an archived, persistent, or DOI-minted version of the repository (e.g., via Zenodo), which is important for long-term citability and reproducibility in a bioinformatics context.

Recommendations

- Add a configuration file (e.g., config.lua or params.lua) exposing all key simulation parameters (pheromone evaporation rate, ant count,

decay constants, food quantity, random seed) with inline documentation of their roles and value ranges

- Document the biological or algorithmic basis for parameter values, citing relevant ant colony optimization literature (e.g., Dorigo & Gambardella 1997)
 - Create a dependency manifest pinning the Love2D version (e.g., a .luarocks or explicit version table in README) and document the version and license of each library in the 'libs' folder
 - Archive the repository on Zenodo and obtain a DOI to enable persistent, citable reference
 - Add a GitHub Actions CI workflow that at minimum verifies the Lua code passes a syntax check (e.g., luac or luacheck)
 - Explain the role of the Python and Makefile components in the README
 - Link or describe the original ant colony simulation that inspired the remake, enabling methodological traceability
 - Add a random seed parameter that can be set by the user to enable deterministic replay of simulation runs
 - Expand the README to include a brief description of the simulation algorithm (stigmergy, pheromone dynamics, foraging behavior) so users can understand what is being simulated
-

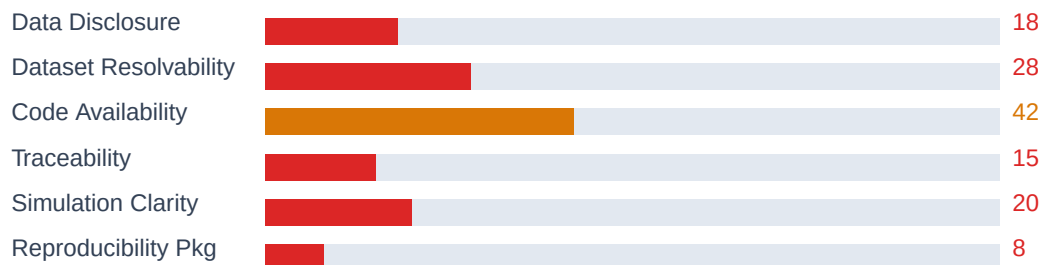
swarm-abc — Artificial Bee Colony Simulation

Source: <https://github.com/darwiiiish/swarm-abc>

Evaluated: June 3, 2026

22.0

Overall /100



Summary

This repository contains a bare-minimum public GitHub deposit of an Artificial Bee Colony simulation implemented in JavaScript and HTML, with no README, no license, no dependency manifest, no tests, no CI, and a single commit history. Because this is a self-contained generative simulation with no external datasets, scoring pivots to disclosure of generative inputs (parameters, initial conditions, rules), none of which are documented in any accessible file. The code is technically public but practically non-reproducible without manual inspection of undocumented source files, and the absence of any documentation, license, or versioning severely limits its utility to the research community.

Positive Findings

- Source code for the ABC simulation is publicly accessible on GitHub at <https://github.com/darwiiiish/swarm-abc>
- Repository contains a JavaScript implementation (abc.js) that constitutes the core algorithm
- An explanation.html file is present, which may contain some inline description of the simulation
- Repository was pushed recently (2026-05-23), indicating it is not an abandoned or stale deposit
- The simulation is self-contained (no external dataset dependencies), which simplifies the reproducibility surface

Gaps Identified

- No README file is present; there are zero instructions for running, configuring, or understanding the simulation
- No license file detected, meaning legal reuse and reproduction rights are undefined
- Simulation parameters (colony size, limit threshold, number of food sources, iteration count, etc.) are not documented in any accessible configuration or documentation file
- No dependency manifest (package.json or equivalent) is present, so runtime environment requirements are unspecified
- No versioned releases or tags exist; only a single commit, making it impossible to pin to a specific state
- No tests or CI configuration are present, so correctness of the implementation cannot be automatically verified
- Contents of the assets folder are undescribed and their role in the simulation is unknown
- No archived snapshot (e.g., Zenodo DOI) exists, so long-term citability is not guaranteed
- No random seed specification or initialization conditions are documented
- No supplementary material or parameter tables are linked from the repository
- [Critic] No evidence of version history, commits, or maintenance activity to confirm the repository is actively maintained or reflects the code described in the paper.
- [Critic] No evidence that the code has been tested, validated, or produces results consistent with those reported in the paper.
- [Critic] No license information confirmed, which affects reproducibility and reuse rights.
- [Critic] No evidence linking the GitHub repository owner ('darwiiiish') to the paper's authors, leaving provenance of authorship unverified.
- [Critic] No evidence of documentation (e.g., README) explaining how to run the simulation, making reproducibility uncertain.
- [Critic] The repository language breakdown (47.9% HTML, 37.7% JS, 14.4% CSS) suggests a browser-based visualization tool, but there is no evidence this constitutes a rigorous scientific implementation suitable for the bioinformatics claims made in the paper.
- [Critic] No evidence of input data, parameter configurations, or example outputs that would allow independent replication of the paper's results.

Recommendations

- Add a comprehensive README.md documenting the purpose of the simulation, how to run it (e.g., 'open index.html in a browser'), key parameters and their default values, and expected outputs
- Create and commit a LICENSE file (e.g., MIT or Apache 2.0) to clarify reuse rights
- Document all ABC algorithm parameters (number of employed/onlooker/scout bees, food source count, limit value, maximum cycles) either in the README or in a dedicated config file
- Tag a versioned release on GitHub (e.g., v1.0.0) and archive it to Zenodo to obtain a citable DOI
- Describe the contents and role of the assets folder in the README
- Add a package.json or equivalent manifest specifying any runtime dependencies and the Node/browser version used
- Specify random seed handling in the code and document how to set seeds for deterministic reproduction
- Add at minimum a simple integration test or example output (e.g., expected convergence behavior for a fixed seed) to allow verification of correct execution
- Consider adding inline code comments explaining non-obvious algorithmic choices in abc.js

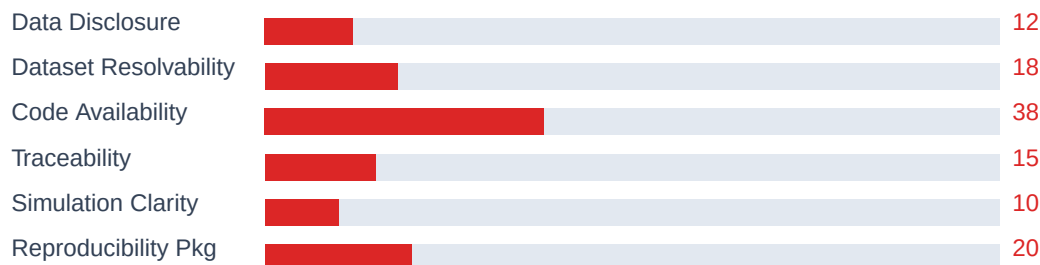
Hardware-Accelerated Ant Colony Swarm (C++)

Source: <https://github.com/matheuslosilva/Hardware-Accelerated-Ant-Colony-Based-Swarm-System>

Evaluated: June 3, 2026

22.0

Overall /100



Summary

This repository presents a hardware-accelerated ant colony swarm simulation using CUDA and OpenGL, but is severely lacking in reproducibility infrastructure. The GitHub API confirms zero releases, zero tags, no license, no dependency manifest, no CI, and a missing/empty README at the root level, making this closer to a bare code dump than a reproducible research artifact. While the source code is publicly accessible and build instructions are described, the evolutionary algorithm parameters, swarm size ranges tested, simulation configuration, and experimental results are entirely undisclosed. No formal paper, data availability statement, or archived snapshot (e.g., Zenodo DOI) accompanies the repository, leaving the scientific claims unverifiable.

Positive Findings

- Source code for the simulation system is publicly available on GitHub under a public repository
- Build instructions are provided (git clone with --recursive, apt-get dependencies, make/make run workflow)
- External dependencies GLFW and GLM are identified by name with download links
- CUDA and OpenGL are identified as the hardware acceleration frameworks used
- Two external OpenGL/CUDA tutorial repositories are cited as development references
- The repository uses git submodules for at least some dependencies, suggesting some dependency management
- A Documentation folder exists in the repository file structure
- The high-level problem domain (search and rescue, ant colony optimization, evolutionary algorithm) is described in the README

Gaps Identified

- No software license is present (confirmed by GitHub API), creating legal ambiguity for reuse
- No versioned releases or tags exist; reproducibility is tied only to the current HEAD of main branch
- README is flagged as missing or empty by the GitHub API, contradicting prose-level build instructions
- No dependency manifest (requirements.txt, CMakeLists.txt with pinned versions, or equivalent) is present
- No CI/CD pipeline or automated tests are configured
- No Zenodo/DOI archive exists for long-term preservation
- Evolutionary algorithm parameter settings, ranges, population size, and termination criteria are completely undisclosed
- Swarm size values or ranges studied are not specified anywhere
- No simulation configuration files, initial condition specifications, or random seeds are provided
- No experimental results, output datasets, or benchmarking data are available or linked
- No formal paper or preprint is linked to contextualise the scientific claims
- The Documentation folder content is not described or linked in the README
- Git submodule identities and pinned versions are not described
- No data availability statement of any kind is present
- [Critic] No empirical benchmarks or scalability experiments are cited to support the claim that the system can simulate 'a large number of agents simultaneously' — the maximum agent count, simulation speed, or comparison to CPU baselines is unspecified.
- [Critic] No validation of the ant colony optimization algorithm's correctness is provided (e.g., convergence tests, comparison to known ACO results on standard benchmarks).
- [Critic] The GitHub repository's activity status, completeness, and whether the code compiles and runs successfully are not verified — public availability does not imply functional or reproducible code.
- [Critic] No versioning or archival snapshot (e.g., Zenodo DOI) is provided for the GitHub repository, meaning the cited code could change or be deleted after publication.
- [Critic] CUDA version compatibility and GPU hardware requirements are not specified, creating a reproducibility gap for users attempting to run the system.
- [Critic] No description of the ant colony algorithm variant used (e.g., ACS, MMAS, AS) is provided, making it impossible to assess algorithmic provenance or correctness independently.

Recommendations

- Add a software license (e.g., MIT or GPL) to enable legal reuse and citation
- Create a tagged release (e.g., v1.0.0) and archive to Zenodo to obtain a citable DOI
- Provide a complete, accurate README with: parameter tables for the evolutionary algorithm, swarm size ranges tested, environment

dimensions, and pheromone decay constants

- Include a dependency manifest with pinned versions (e.g., a CMakeLists.txt specifying exact CUDA toolkit version, GLFW version, GLM version)
 - Add a configuration file (JSON/YAML/INI) that captures all simulation parameters and serves as the single source of truth for reproducing reported experiments
 - Disclose random seeds used for all reported simulation runs
 - Publish experimental results (e.g., as CSV files or a Jupyter notebook) alongside the code
 - Set up GitHub Actions CI to at least compile-check the code on push
 - Expand the Documentation folder and link its contents explicitly from the README
 - Link or upload any associated paper, report, or preprint so that code-to-claim traceability can be established
 - Describe git submodule contents and their pinned commits explicitly in the README
-

